HROne Employee Attendance & Analytics API

Project Analysis, System Design & Step-by-Step Development Plan

Backend Python 3.11+ · FastAPI	**Database** MongoDB 6.0+
API contract OpenAPI 3.0.3	**Architecture** Single-file app: app/main.py

Objective

Build a reliable, testable REST API for employee attendance capture, manual corrections with audit history, and MongoDB-powered HR analytics. The priority is exact contract compliance—not unnecessary features.

Assessment deadline stated in the kit: 11:59 PM IST, 15 October 2026.

This plan is based on the supplied assessment files and API contract. Estimates are planning guides, not guarantees.

1. Project Analysis

The assignment is more than a CRUD exercise. It evaluates business-rule correctness, concurrency, MongoDB aggregation, query performance, code review, and the candidate's ability to explain engineering decisions.

Core capabilities

Attendance capture: punch-in and punch-out with correct attendance-date attribution, lateness, overtime, work-hour rounding, and half-day calculations.

Regularization: validate manual corrections, recompute derived fields, and append a before/after audit entry without losing concurrent edits.

Analytics: monthly employee summaries, cross-collection department summaries, a tied late leaderboard, and a gap-filled daily department trend.

Operations: database-aware health check, required indexes, efficient pagination, and an explain endpoint that returns MongoDB execution statistics.

Main risks

Risk	Why it matters
IST and overnight shifts	Punch-out can occur the next calendar day while belonging to the shift's original attendance date.
Concurrent writes	Check-then-insert is not sufficient to prevent duplicate punch-ins; punch-out and corrections also need atomic protection.
Rounding and boundaries	Rules specify strict thresholds, whole-minute floors, and half-up rounding rather than Python's default round behavior.
Analytics edge cases	No-log employees, mid-month joiners, weekends, zero headcount, ties, and missing dates affect results.
Query performance	The grader uses roughly 100,000 attendance records and checks query plans for index scans.

Scope constraints

Keep all application code in app/main.py. Required repository files: app/main.py, requirements.txt, REVIEW.md, DECISIONS.md, and README.md. Do not submit .env files, secrets, a Dockerfile, a virtual environment, or data dumps.

2. System Design

Recommended logical flow

HTTP client / API tester
JSON requests and query parameters

↓

FastAPI routes + Pydantic validation
Input validation, status codes, response serialization

↓

Business logic helpers
IST conversion, shift calculations, half-up rounding, audit history

↓

MongoDB operations
Atomic writes, indexes, filters, sorting, pagination, aggregation pipelines

Single-file does not mean unstructured: group related helpers and endpoint handlers by clear comment sections, reuse common validation and serialization logic, and avoid duplicating calculations.

3. Database Design and Indexes

The kit defines two collections. Preserve its schema and do not expose MongoDB _id as the public employee or attendance identifier.

employees

Fields: emp_code, name, email, department, shift_start, shift_end, joined_on, created_at. emp_code is the client-provided immutable business identifier.

attendance_logs

Fields: emp_code, date, status, punch_in, punch_out, work_hours, late_minutes, overtime_minutes, half_day, history.

Collection	Candidate index	Purpose
employees	Unique {emp_code: 1}	Uniqueness and employee lookups.
employees	{department: 1, emp_code: 1}	Department filter and sorted listing.
employees	{department: 1, joined_on: 1}	Department headcount and joining-date queries.
attendance_logs	Unique {emp_code: 1, date: 1}	Prevents duplicate daily records; supports employee/date lookup.
attendance_logs	{date: -1, emp_code: 1}	Date sorting and attendance-list access.
attendance_logs	{emp_code: 1, date: 1, status: 1}	Employee monthly filtering and status queries.

These are candidate indexes. Validate each actual query plan using the explain endpoint; overlapping indexes may be unnecessary, and the final set must support the evaluator's queries.

4. API Inventory and Priority

Priority	Endpoint	Main concern
P0	GET /health	MongoDB ping; return 503 when unavailable.
P0	POST /employees	Validation and database-enforced uniqueness.
P0	GET /employees	Correct filter count, pagination and sorting.
P0	POST /attendance/punch-in	IST attendance date, lateness and concurrency.
P0	POST /attendance/punch-out	Atomic closure and overnight shifts.
P0	GET /attendance	Indexed filters, deterministic sort and pagination.
P0	PATCH /attendance/{emp_code}/{date}	Recompute derived fields and append audit history.
P1	GET /analytics/employees/{emp_code}/monthly	Monthly summary using aggregation.
P1	GET /analytics/departments/summary	Cross-collection headcount and statistics.
P1	GET /analytics/leaderboard/late	Competition ranks and cutoff ties.
P1	GET /analytics/departments/{department}/trend	Date gap filling and seven-day moving average.
P1	GET /admin/explain/{endpoint}	Actual query/pipeline explain with execution stats.

P0/P1 describe implementation order, not optional features. All 12 endpoints are required.

5. Step-by-Step Development Plan

Step 1 — Inspect contract and starter (15–20 min)

Read openapi.yaml, DATA_MODEL.md, and app/main.py. Map every endpoint to inputs, outputs, status codes, and database operations. Record concrete starter defects in REVIEW.md.

Step 2 — Setup, validation, indexes (20–25 min)

Configure MONGO_URI and MONGO_DB, initialize MongoDB at startup, create indexes idempotently, define Pydantic models, and add timestamp/serialization helpers. Make /health ping MongoDB.

Step 3 — Fix employee and attendance lists (25–30 min)

Correct page offset to (page - 1) * page_size, count the filtered set, sort deterministically, push filters/sort/pagination into MongoDB, validate date ranges, and omit _id from responses.

Step 4 — Attendance rules (35–40 min)

Implement epoch-millisecond validation, second truncation, UTC storage, IST attendance date, overnight attribution, late/overtime/work-hours/half-day helpers. Use a unique index for punch-in and conditional atomic update for punch-out.

Step 5 — Manual corrections (30–35 min)

Validate final status/times, recompute derived fields, record only actual changes, append one history entry atomically, and reject stale concurrent edits with 409.

Step 6 — Four analytics pipelines (60–75 min)

Implement employee monthly summary; department summary across employees and logs; late leaderboard with competition ranking; and department trend using database-side date gap filling and moving average.

Step 7 — Explain endpoint (20–30 min)

Use the exact endpoint query/pipeline and filters with executionStats. Verify IXSCAN and no COLLSCAN. Adjust indexes and query shape based on actual plans.

Step 8 — Tests and submission (20–25 min)

Run edge-case and concurrency tests, finish REVIEW.md, DECISIONS.md, README.md, verify clean-clone startup and Git status, push public repository, submit the link.

6. Exact Attendance Rules to Implement

Rule	Required behavior
R1 — Time zone/date	Shift times and dates use IST (UTC+05:30). Attendance date is the IST date of punch-in, except early-morning punches before shift_end for overnight shifts belong to the prior shift-start date. Truncate instants to whole seconds before calculation/storage.
R2 — Lateness	Grace is 10 minutes. Strictly more than 10 minutes late triggers lateness; report floor minutes since shift start, not after grace.
R3 — Overtime	Floor minutes after shift end; count only when elapsed overtime is at least 30 minutes.
R4 — Work hours	Elapsed seconds / 3600, rounded to two decimals using half-up rounding. Null until punch-out.
R5 — Half day	After punch-out, true when rounded work_hours is below 4.50.
R6 — Presence	PRESENT, WFH, ON_DUTY are presence statuses. ABSENT and LEAVE are not.
R7 — Working days	Monday–Friday; start counting from joined_on for mid-month joiners. Weekend records still contribute to late/overtime totals.
R8 — Report rounding	Two decimals half-up; rates use four decimals half-up.
R9 — Headcount	Employees joined on or before period end count even without logs; employees joining later do not.
R10 — Pagination	Page starts at 1, page_size maximum is 100, default 20, total reflects applied filters.

7. Concurrency-Safe Write Design

Punch-in

1. Validate employee and calculate the attendance date. 2. Attempt insert under the unique (emp_code, date) index. 3. Return 201 on success. 4. Convert duplicate-key error into 409. Do not rely on a prior find_one check.

Punch-out

1. Find the most recent eligible open record with punch_in <= punch-out instant. 2. Validate the timestamp is after punch-in and within 24 hours. 3. Atomically update only if punch_out is still null. 4. One simultaneous request wins; the other cannot overwrite the result.

Regularization

Use an atomic update with a prior-state/version condition. Append the audit entry in the same document update. If the record changed since it was read, return 409 rather than losing another correction's history.

8. Analytics Pipeline Design

A. Employee monthly summary

Filter by employee and month, then aggregate totals. Count weekday present-days (half day = 0.5), leave records, late records, late minutes and overtime. Working days begin no earlier than joined_on. Return attendance_pct as null when working_days is zero.

B. Department monthly summary

Start from eligible employees, retain employees with no logs, and join monthly attendance. Exclude employees joining after month-end. Average work_hours across qualifying present-status records with non-null work_hours—not across per-employee averages. Omit departments with zero eligible headcount.

C. Late leaderboard

Use $setWindowFields with $rank or equivalent MongoDB competition ranking. Ties share a rank and the next rank is skipped. Filter department before ranking, rank only employees with positive late totals, and apply limit to rank so cutoff ties are retained.

D. Department trend

Validate the inclusive range (maximum 92 days). Aggregate daily values, fill missing dates in MongoDB using $densify or an equivalent database-side strategy, compute date-specific headcount, then calculate attendance_rate and the seven-row moving average. Do not build the date series in a Python loop.

Important: $densify fills missing dates but does not calculate historical headcount by itself. Each generated date still needs headcount based on joined_on.

9. Test Strategy

Area	Case	Expected result
Employee	Concurrent duplicate employee creation	Exactly one 201; others 409.
Pagination	Page 1, page_size 20	First 20 filtered records and correct filtered total.
Timestamp	Seconds-looking timestamp	422.
Lateness	09:30 shift; punch at 09:40:00	0 late minutes.
Lateness	09:30 shift; punch at 09:40:01	10 late minutes.
Overnight	22:00–06:00 shift, punch-out next day	Correct attendance date and shift end.
Punch-out	Two simultaneous punch-outs	Exactly one success; no overwrite.
Correction	No-op correction	422.
Correction	Concurrent edits	No lost history; conflict may return 409.
Analytics	Employee with no logs	Included in eligible headcount.
Analytics	Weekend attendance	Excluded from monthly present_days; late/overtime still counted.
Leaderboard	Tie at rank cutoff	All employees whose rank <= limit are returned.
Trend	Missing weekday with positive headcount	Date included and rate is 0.0.
Explain	Large dataset	Index scan present; no collection scan.

10. Repository and Submission Checklist

Required repository layout

```
hrone-attendance-api/
■■■ app/
■ ■■■ main.py
■■■ requirements.txt
■■■ REVIEW.md
■■■ DECISIONS.md
■■■ README.md
```

Run from repository root

```
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Final checklist

■ All 12 endpoints implemented and tested against openapi.yaml.

■ Indexes created automatically at startup; MongoDB configuration comes from environment variables.

■ REVIEW.md lists concrete defects, symptoms, fixes, and examined non-defects.

■ DECISIONS.md answers all five template questions in the candidate's own words.

■ README.md includes setup, environment variables, run command, and known omissions.

■ No .env, secrets, Dockerfile, virtual environment, or prohibited data dumps committed.

■ Clean clone installs and starts; repository is public and submission link is sent through the provided form.

11. Prioritization if Time Is Short

First: startup, indexes, validation, starter defects, attendance calculations and concurrency.

Second: all four analytics pipelines and the explain endpoint; these are mandatory even if more difficult.

Third: tests, documentation, clean repository, and submission. Avoid adding a frontend, authentication system, or other unrequested features.

The strongest submission is one where code, documented decisions, and observed behavior agree. Be prepared to explain your atomic writes, index choices, aggregation stages, and edge-case tests in the live walkthrough.